UTN FRLP

Desarrollo de Software 2026

APIs REST

Clase #15 - 3 de agosto de 2026

Matías Corti - Martina García Améndola



Dónde quedamos

Antes del receso dejamos andando:

- `docker compose up` → `api` + `db`
- `GET /libros` respondiendo con nuestros libros hardcodeados
- Probado con REST Client, desde un `api.http` commiteado

Pendientes de C14	**Lo resuelve**
Todo en un solo archivo	← HOY
Datos hardcodeados	C16 · PostgreSQL + Prisma
Nadie valida lo que llega	C17 · Zod en el back
Cualquiera accede a todo	C18 · Auth con JWT
El front sigue leyendo el .json	C19 · Conexión front ↔ back

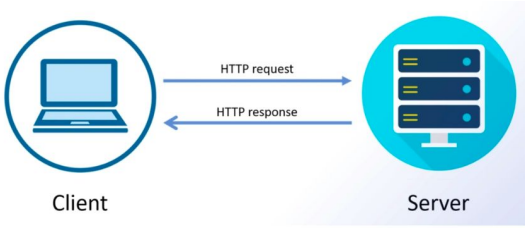
Temas de hoy

- **API REST:** restricciones, verbos, códigos de estado, cómo se nombra una ruta
- **Diseño de API:** de la tabla de operaciones de C13 a la tabla de endpoints
- **Estructura en capas:** routes, controllers, services, types
- **Práctica:** CRUD completo de libros, en capas, corriendo en Docker



HTTP

EL IDIOMA UNIVERSAL DE INTERNET



🔄 REQUEST - RESPONSE

Cliente envía petición → Servidor procesa → Servidor responde

MÉTODOS HTTP Y SU SEMÁNTICA

- **GET** Solicitar recursos
- **POST** Crear recursos
- **PUT** Actualizar/reemplazar completo
- **DELETE** Eliminar recursos
- **PATCH** Actualización parcial

CÓDIGOS DE ESTADO

- **2xx** Éxito
  200 OK, 201 Created
- **4xx** Error del cliente
  400 Bad Request, 404 Not Found, 401 Unauthorized
- **5xx** Error del servidor
  500 Internal Server Error



REST



REST: QUÉ ES Y QUÉ NO

¿Qué es? REST (*REpresentational State Transfer*) es un **estilo de arquitectura** para APIs sobre HTTP: convenciones sobre cómo nombrar los recursos y qué se les puede hacer.

- **Recurso** = la cosa: un libro, un autor, un pedido. La **URL** lo identifica, el **verbo** dice qué hacerle
- **¿Por qué?** Si todos usamos las mismas convenciones, cualquiera entiende tu API sin leer tu código: por eso se puede usar la API de Mercado Libre o de GitHub sin preguntarle nada a nadie

Qué NO es REST:

- ❌ **No** es "cualquier API que devuelve JSON"
- ❌ **No** es un protocolo ni una librería: no se instala, no se importa
- ❌ **No** es una norma con policía: es un estilo, se cumple más o menos



RESTRICCIONES QUE NOS IMPORTAN

Las cuatro que vas a ver hoy en tu propia API:

Restricción	Qué significa	Dónde la vas a ver hoy
Cliente-servidor	El que pide y el que responde son independientes	Tu React no sabe cómo está hecha tu API, y viceversa
Interfaz uniforme	Las mismas convenciones para todos los recursos	`/api/libros` y `/api/autores` se operan igual
Stateless	Cada request trae todo lo necesario. El servidor no se acuerda del anterior	No hay "sesión": el `:id` viaja en la URL, siempre
Cacheable	Las respuestas pueden guardarse para no volver a pedirlas	Se usa mucho en producción. Lo nombramos y seguimos



Cómo se nombra una ruta

Cuatro reglas:

1. **Sustantivos, no verbos**
   → `/api/libros`, no `/api/traerLibros`. El verbo lo pone HTTP

2. **Plural para la colección**
   → `/api/libros` (la lista) y `/api/libros/3` (uno)

3. **El `:id` identifica, no describe**
   → `/api/libros/:id`

4. **Filtrar y ordenar va en el query string, no en la ruta**
   → `/api/libros?disponible=true`

La regla 4 es la que más se rompe: **cada filtro nuevo no es una ruta nueva.**

```
/api/libros                    → la colección
/api/libros/3                  → el libro 3
/api/libros?disponible=true    → la colección, filtrada
/api/autores/2/libros          → los libros del autor 2
```



Verbos HTTP

Idempotente = hacerlo diez veces deja el sistema igual que hacerlo una vez.

`DELETE /api/libros/3` diez veces → el libro 3 no está, **igual**.
`POST /api/libros` diez veces → **diez libros**, distinto.

Verbo	Qué hace	Idempotente	Lleva body
GET	Leer	✅ Sí	❌ No
POST	Crear	❌ No	✅ Sí
PUT	Reemplazar completo	✅ Sí	✅ Sí
PATCH	Modificar parcial	❌ No	✅ Sí
DELETE	Borrar	✅ Sí	❌ No

PUT manda el recurso **completo** y lo reemplaza.
PATCH manda **sólo lo que cambia** → hoy PUT
PATCH vuelve en **C17**, con Zod.

🔢

Códigos de estado

Los **seis** que vas a usar:
2xx bien, **4xx** error del cliente, **5xx** error en el servidor

Código	Nombre	Cuándo lo devolvés
200	OK	GET o PUT que salió bien
201	Created	POST que creó algo (y devolvés lo creado)
204	No Content	DELETE que salió bien. Sin body
400	Bad Request	Datos mal del cliente → hoy no lo emitimos (C17 con validaciones)
404	Not Found	El recurso no existe
500	Server Error	Explotó nuestro código

```
const res = await fetch(url);              // Nuestro código de C11
if (!res.ok) throw new Error('Error al cargar los datos');
// res.ok es 2xx. Ese status lo decide el servidor:hoy son ustedes.
```



Anti-patrones

El patrón del error es siempre el mismo: **el verbo metido en la URL.**

Si tu ruta tiene un verbo adentro, ese verbo sobra.

❌ Así no	✅ Así
GET /api/traerLibros	**GET** /api/libros
POST /api/libros/borrar/3	**DELETE** /api/libros/3
GET /api/librosDisponibles	**GET** /api/libros?disponible=true
POST /api/actualizarPrecio	**PUT** /api/libros/3
GET /api/libros/crear	**POST** /api/libros

🔁 Mini-cierre: REST

Lo que nos llevamos:

Concepto	En una línea
REST	Estilo de arquitectura: convenciones sobre recursos y verbos
Recurso	La cosa. La URL la identifica, el verbo dice qué hacerle
Stateless	Cada request trae todo; el servidor no se acuerda del anterior
Idempotente	Repetirlo deja el sistema igual
Verbo en la URL	El error más común. Si está, sobra
2xx / 4xx / 5xx	Bien / culpa del cliente / culpa tuya
res.ok	El status que ahora decidís vos



Diseño de API



El contrato, del otro lado

Previamente dijimos que una API es **un contrato** que define tres cosas. Ahora que el contrato lo escriben ustedes, esas tres cosas tienen nombre técnico.

Y el contrato se escribe **antes** del código: diseñar después de codear no es diseñar, es documentar.

Lo que definimos	**Lo que es**
Qué se puede pedir	Qué rutas existen
Cómo se debe pedir	Verbo + qué va en el body
Qué se espera como respuesta	Status code + forma del JSON



De operaciones a endpoints

En **C13** armaron una tabla como esta, revisando pantallas.
No es la API: es la materia prima.

Podemos encontrar:

- Varias operaciones → **un endpoint**
- Una operación → **más de un endpoint**
- endpoints que **ninguna pantalla pidió** (borrar)

Traducir una tabla en la otra **es nuestro diseño de API.**

Pantalla	**Operación**	**Entidad**
Home	Ver destacados	Libro
Catálogo	Listar libros	Libro
Catálogo	Buscar / filtrar	Libro
Detalle	Ver un libro	Libro
Alta	Crear un libro	Libro
Edición	Editar un libro	Libro



REST O NO REST

De a una ruta por vez. Votan a mano alzada:
✅ **está bien** y ❌ **está mal.**

El que vota ❌ dice por qué.

⏱️ **6 minutos** - sin computadora - sin internet

```
 1.  GET     /api/libros/3
 2.  POST    /api/libros/3/borrar
 3.  GET     /api/libros?autor=Cortazar
 4.  PUT     /api/libro/3
 5.  POST    /api/libros
 6.  GET     /api/borrarLibro/3
 7.  DELETE  /api/libros
 8.  GET     /api/autores/2/libros
 9.  POST    /api/libros/buscar
10.  PATCH   /api/libros/3
```



La tabla de endpoints: Librería

6 operaciones → 5 endpoints

Este es el documento que implementan hoy. El **400** con asterisco está en el contrato pero hoy **no lo emitimos** (no hay quién valide lo que llega, lo haremos en **C17**).

Método	Ruta	Operaciones que cubre	Body	Éxito	Errores
GET	/api/libros	Listar, Buscar, Destacados	—	200	500
GET	/api/libros/:id	Ver un libro	—	200	404
POST	/api/libros	Crear un libro	Libro sin id	201	400* 500
PUT	/api/libros/:id	Editar un libro	Libro sin id	200	404 400*
DELETE	/api/libros/:id	Borrar un libro	—	204	404



Taller: tabla de endpoints de su TP

Por grupo, sobre su propio TP:

1. Abran su Documentación / 05 - Diseño de API, donde está la tabla de operaciones de C13

2. **Debajo, tabla nueva.** No toquen la de operaciones: quedan las dos → Método | Ruta | Operaciones que cubre | Body | Éxito | Errores

3. Empiecen por **una sola entidad**, la más importante de su dominio. Mínimo **5 endpoints**

4. Cuando dos operaciones caigan en el mismo endpoint, **pónganlas juntas en la misma fila**

⏱️ **15 minutos** en clase, se completa en [AG-C15]

✅ **Listo cuando:** su entidad principal tiene sus 5 endpoints, cada uno con verbo, ruta, status de éxito y status de error.

Al cierre: **cada grupo lee una fila** y entre todos lo validamos.



Estructura en capas



El problema

El index.ts de C14: **60 líneas**, 2 entidades, 3 endpoints. Hoy le sumamos **8 endpoints más** → ~200 líneas en el mismo archivo, y todavía falta validar, persistir y autenticar.

Tres preguntas:

- ¿Qué archivo abrís para cambiar **cómo se busca** un libro?
- ¿Pueden **dos personas de tu grupo** trabajar acá a la vez?
- ¿Cómo probás la lógica **sin levantar un servidor**?

```
// backend/src/index.ts de C14
import express from "express";
const app = express();
const PORT = 3000;

app.get("/", (_req, res) => { ... });       // ← configuración

interface Libro { ... }                     // ← tipos
interface Autor { ... }

const libros: Libro[] = [ ... ];            // ← datos
const autores: Autor[] = [ ... ];

app.get("/libros", ( req, res) => { ... }); // ← rutas + lógica + HTTP
app.get("/autores", (_req, res) => { ... });
```



Por qué capas

El problema no es que sea **largo**, es que hace **cinco cosas a la vez:**

- Decide qué rutas existen
- Traduce HTTP (leer el `:id`, elegir el status)
- Guarda y busca los datos
- Define los tipos
- Levanta el servidor

Cuando todo está junto:

- ❌ Para cambiar **cómo se busca** un libro, tocás el mismo archivo que atiende HTTP
- ❌ **Dos personas no pueden** trabajar en paralelo sin pisarse
- ❌ No podés **testear** la lógica sin levantar un servidor
- ❌ Nadie encuentra nada

Separar en capas: cada archivo tiene una responsabilidad clara, y sabe lo mínimo posible del resto.



Quién hace qué

Capa	Su única responsabilidad	Lo que NO hace
routes	Decir qué verbo + ruta existe, y a qué controller va	No tiene lógica. Ni un if
controllers	Traducir HTTP ↔ dominio: leer params / query / body, elegir el status code	No busca ni guarda datos
services	La lógica y el acceso a los datos	No sabe que existe HTTP. No ve req ni res
types	Las formas de los datos, compartidas entre capas	No tiene lógica

La regla de oro: si el **service** menciona `req`, `res` o un status code, está mal. Es la que más se viola.

ESTRUCTURA ACTUAL

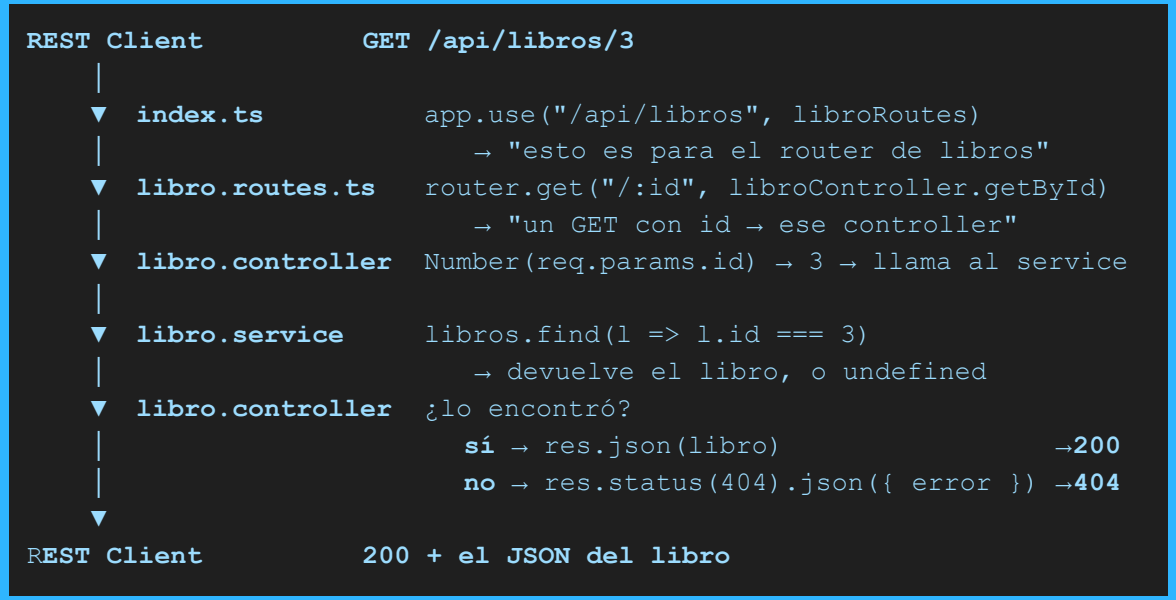
Nombre de archivo: **entidad en singular + rol.**
Cuatro carpetas, no nueve: las que faltan llegan cuando se necesiten.

```
backend/src/
├── types/
│   └── libro.types.ts        ← interface Libro
├── services/
│   └── libro.service.ts      ← los datos y la lógica
├── controllers/
│   └── libro.controller.ts   ← HTTP ↔ service
├── routes/
│   └── libro.routes.ts       ← verbo + ruta → controller
└── index.ts                  ← levanta el server y monta las rutas
```

Carpeta	Llega en	Para qué
config/	C16	La conexión a Prisma
middlewares/	C17 y C18	Validación con Zod, luego auth
utils/	Cuando haga falta	Todavía no hace falta

Ejemplo Completo: DS26-Libreria (C15)

El camino de un request



La ruta en `libro.routes.ts` se escribe **relativa**: `/:id`, no `/api/libros/:id`.

El prefijo lo pone el `app.use()`, una sola vez.



EXPRESS.JSON() LA LÍNEA QUE HACE EXISTIR EL BODY

Hasta hoy sólo **leímos** datos. Con POST y PUT, el cliente **manda** un JSON.

- Express **no** parsea el body por defecto: hay que pedírselo
- `express.json()` lee el body, lo parsea y lo deja en `req.body`
- Sólo lo hace si el request trae el header `Content-Type: application/json`
- Va **antes** de las rutas. Si va después, no se ejecuta

Es un **middleware**: algo que corre *antes* de tu handler. El concepto formal lo vemos en **C17**, cuando lo necesitemos para validar.

```
// backend/src/index.ts
app.use(express.json());   // ← sin esta línea, req.body es undefined
app.use("/api/libros", libroRoutes);
```

⚠️ El error que van a tener hoy: `req.body` viene `undefined` o `{}`. Casi siempre es una de esas dos líneas.

🔁 Mini-cierre: capas

Lo que nos llevamos — la chuleta de los próximos 75 minutos:

Concepto	**En una línea**
routes	Verbo + ruta → controller. Nada más
controllers	Traducen HTTP y eligen el status code
services	Lógica y datos. No conocen HTTP
types	Las formas, compartidas
app.use("/api/libros", router)	Monta el router; las rutas de adentro son relativas
express.json()	Sin ella, req.body no existe. Va antes de las rutas
La regla de oro	Si el service ve req, res o un status, está mal



PRÁCTICA



Paso 1: types y service

[Ejemplo Completo: DS26-Libreria (C15)]

El `types` sale tal cual del `index.ts`. El `service` se lleva **los datos y la lógica, sin HTTP.**

⏱️ **12 minutos**
✅ **Listo cuando:** `docker compose up` sigue andando y `GET /libros` responde **200**, igual que antes.

```ts
// src/types/libro.types.ts
export interface Libro {
  id: number; titulo: string; autor: string;
  precio: number; imagen: string; disponible: boolean;
}
```

```ts
// src/services/libro.service.ts
const libros: Libro[] = [ /* los 10 de C14 */ ];
let proximoId = 11;

export function findAll(disponible?: boolean): Libro[] {
  if (disponible === undefined) return libros;
  return libros.filter(l => l.disponible === disponible);
}

export function findById(id: number): Libro | undefined {
  return libros.find(l => l.id === id);
}
```

Paso 2: controller y routes

El controller traduce HTTP y elige el status.
Las routes son sólo verbo + ruta → controller.

⏱️ **13 minutos**
✅ **Listo cuando:** GET /api/libros → **200** y GET /api/libros/999 → **404**

⚠️ La ruta cambió: ahora es **/api/libros**. La vieja /libros ya no existe → actualicen el api.http.

```
// src/controllers/libro.controller.ts
export function getById(req: Request, res: Response) {
  const libro = libroService.findById(Number(req.params.id));
  if (!libro) return res.status(404).json({ error: "Libro no
encontrado" });
  return res.json(libro);
}
```

```
// src/routes/libro.routes.ts
const router = Router();
router.get("/", libroController.getAll);
router.get("/:id", libroController.getById);
export default router;
```

```
// src/index.ts - queda en ~20 líneas
app.use(express.json());
app.use("/api/libros", libroRoutes);
app.use("/api/autores", ...);   // autores, todavía sin capas
```

Ejemplo Completo: DS26-Libreria (C15)



Paso 3: POST el primero con body

create() en el **service**: leer req.body y elegir el status en el **controller** · router.post("/", ...) en las **routes**.

Omit<Libro, "id"> = un Libro sin id: **el id lo pone el servidor, no el cliente.**

⏱️ **15 minutos**
✅ **201** + el libro creado con su id; después GET /api/libros lo lista.

```
export function create(datos: Omit<Libro, "id">): Libro {
  const nuevo: Libro = { id: proximoId++, ...datos };
  libros.push(nuevo);
  return nuevo;
}
```

```
### Crear un libro
POST {{baseUrl}}/api/libros
Content-Type: application/json

{ "titulo": "Rayuela", "autor": "Julio Cortázar", "precio": 7000,
  "imagen": "https://...", "disponible": true }
```

Ejemplo Completo: DS26-Libreria (C15)



Paso 4: PUT y DELETE, con sus 404

En el **service**: `update(id, datos)` devuelve el libro actualizado o `undefined`; `remove(id)` devuelve `true` o `false`.
En el **controller**: `undefined` / `false` → **404**.

```
export function remove(req: Request, res: Response) {
  const ok = libroService.remove(Number(req.params.id));
  if (!ok) return res.status(404).json({ error: "Libro no encontrado" });
  return res.status(204).send();    // 204 = sin body. NO .json()
}
```

⏱️ **15 minutos**
✅ **Listo cuando** las cuatro dan esto:

Request	**Status**
PUT /api/libros/1 con el body completo	200
PUT /api/libros/999	404
DELETE /api/libros/1	204 y respuesta vacía
DELETE /api/libros/1 otra vez	404

Ejemplo Completo: DS26-Libreria (C15)



Paso 5: api.http completo + commit

El `.http` es la **documentación ejecutable** de su API. Se commitea.

`@baseUrl` se define **una vez** arriba: el día que cambie el host, es una línea.

⏱️ **8 minutos**
✅ **Listo cuando:** los requests corren de arriba abajo con los status esperados, y está **commiteado y pusheado**.

```
@baseUrl = http://localhost:3000

### Listar libros
GET {{baseUrl}}/api/libros

### Sólo los disponibles
GET {{baseUrl}}/api/libros?disponible=true

### Un libro
GET {{baseUrl}}/api/libros/1

### Un libro que no existe → 404
GET {{baseUrl}}/api/libros/999

### Crear · Editar · Borrar
### ...
```

Ejemplo Completo: DS26-Libreria (C15)



¿Terminaste? Extensiones

1. **Autores a capas.** `/api/autores` sigue amontonado en `index.ts`. Hacele sus cuatro archivos.
Es la actividad individual: empezala acá.

2. **Cazá el bug.** Hay un endpoint en el branch `C15` que devuelve el status equivocado. ¿Cuál?

3. **Difícil:** probá `GET /api/libros/abc`. ¿Qué status devuelve? ¿Cuál **debería** devolver? ¿Por qué no lo podemos arreglar hoy?



Lo que falta, falta a propósito

Hoy nos llevamos: REST, un diseño de API documentado, un CRUD completo en capas, corriendo en Docker.

Pendientes de hoy	**Lo resuelve**
Reiniciás el container y los libros se van	C16 · PostgreSQL + Prisma
POST sin titulo devuelve 201 y guarda basura	C17 · Zod en el back
GET /api/libros/abc devuelve 404 en vez de 400	C17 · validación
libro.autor es un texto, no un Autor de verdad	C17 · relaciones
Cualquiera puede borrar cualquier libro	C18 · Auth con JWT
Tu front sigue leyendo libros.json	C19 · conexión front ↔ back

Actividades

Actividad Individual

1º instancia hasta el **09/08/2026**
*2º instancia: **13/09/2026***
(Cierre Bloque 3)

[AI-C15] Capas en Librería

Referencia en el repo de ejemplo branch C15

1. En ds2026-actividades: copiar el sitio de la librería de la Clase 14 en **ejercicios/c15-api-rest/**

2. Pasá **libros** a capas: types/, services/, controllers/, routes/, montado en /api/libros

3. Hacé lo mismo con **autores**: /api/autores con su **CRUD completo** (los 5 endpoints)

4. index.ts no puede tener ni un dato ni un if: sólo levanta el server y monta los dos routers

5. api.http con los ~10 requests, **incluyendo los casos de error** (los 404)

6. Commiteá y avisá en #actividades
   → hilo **[AI-C15] Capas en Librería**

Actividad grupal

Hasta el **09/08/2026** (inclusive)

[AG-C15] Diseño de API + primer CRUD

1. **Documentación:** completen en Documentación / "05 - Diseño de API" la **tabla de endpoints** del taller, **todas las entidades**, no sólo la principal. La tabla de operaciones de C13 queda como está, arriba.

2. **Código:** al menos **un CRUD completo** en el backend del TP, **en capas**, con prefijo /api. Una entidad, los 5 endpoints, los status del contrato.

3. api.http commiteado con los requests de esa entidad, **casos de error incluidos**.

4. Dividir las tareas en el equipo, hacer PRs con reviews cruzados (al menos uno cada alumno) y en trello mover tarjetas correspondientes a "Done"

5. Aviso en #actividades
   → hilo **[AG-C15] Diseño de API + primer CRUD**

⚠️

Antes de la próxima clase

⚠️ **INSTALAR ANTES DE C16**

El aula no tiene internet confiable.

Sin instalación = sin práctica.

```bash
# 1) Agregar las dependencias al proyecto
cd backend
npm install prisma@7.9.1 --save-dev
npm install @prisma/client@7.9.1

# 2) Reconstruir la imagen  ← este paso NO es opcional
cd ..
docker compose build api

# 3) Verificar que quedó adentro del contenedor
docker compose up -d
docker compose exec api npx prisma --version
```

✔ Dudas → canal #consultas de Discord

📅 C16: PostgreSQL + Prisma (10/08/2026)